

מטלה מסכמת – צד שרת

עופר אביעוז, 212052385

פיצ'רים שנבחרו :

1. ממשק מנהל המאפשר לבצע פעולות על ניהול אירועים

תיאור הפיצ'ר :

המשתמש נכנס דרך מסך התחברות ייעודי למנהל, מזין שם משתמש (admin) וסיסמה (1234) ולאחר התחברות מוצלחת מגיע למסך הניהול. במסך הניהול ניתן לשלוט בתוכן האירועים המוצג למשתמשים.

באזור זה ניתן לבצע את הפעולות :

Create Event

Update Event

Delete Event

מימוש :

בצד הלקוח נוספו שני דפים -

AdminLogin.html – מסך התחברות לאיזור הניהול

Admin.html – מסך הניהול עצמו

במסך הניהול קיימים טפסים המאפשרים למנהל להזין פרטי אירוע חדש, לעדכן אירוע קיים לפי מזהה או למחוק אירוע לפי מזהה.

הפיצ'ר משתמש בקריאות API קיימות :

POST /api/event – יצירת אירוע חדש

PUT /api/event/{id} – עדכון אירוע קיים

DELETE /api/event/{id} – מחיקת אירוע

בצד השרת הבקשות מתקבלות ב-EventController, מועברות ל-EventService ומשם ל-EventRepository שמבצע את הפעולות מול מסד הנתונים.

הערה :

בעת מחיקת אירוע, המערכת מוחקת גם את כל ה-Sessions והרישומים הקשורים אליה כדי להימנע מנתונים לא תקינים במסד הנתונים.

2. מסך סטטיסטיקות

תיאור הפיצ'ר :

מציג מידע מסכם על מצב המערכת.

מיועד לתת למשתמש תמונה כללית על כמות האירועים, ה-Sessions, הרישומים במערכת ועוד.

המסך מציג בין היתר :

Total Events, Total Sessions, Average Participants per Session, Upcoming Events,
Events With Most Sessions, Average Sessions Per Event, Total Registrations,
Events By Month

מימוש :

בצד הלקוח נוסף דף – Statistics.html

הדף משתמש ב-JS כדי למשוך נתונים מה-API הקיים, בעיקר מרשימת האירועים וה-Sessions ומחשב מהם נתונים סטטיסטיים.
כל המידע מוצג באופן מרוכז וברור למשתמש.

3. לוח זמנים אישי למשתמש (לא מהרשימה הנתונה)

תיאור הפיצ'ר :

המשתמש מזין את ה-UserId שלו, והמערכת מציגה לו את כל ה-Sessions שאליהם הוא רשום.
המידע המוצג כולל : שם ה-Session, תיאור, שם המרצה/אמן, שעת התחלה, שעת סיום, חדר או במה, תאריך הרשמה.

מימוש :

בצד הלקוח נוסף דף – MySchedule.html

בצד השרת קיימת הקריאה – GET /api/user/{userId}/schedule

הבקשה מתקבלת ב-UserController, עוברת ל-UserService, ומשם ל-UserRepository.
הריפויטורי שולף את הרשומות מהדאטה בייס עבור המשתמש, כולל פרטי ה-Session ומחזיר אותן באופן מסודר.

הנתונים מוחזרים ללקוח באמצעות UserScheduleDTO.

נספח – סיכום נקודות מרכזיות בפרויקט

Weather API עם מנגנון Cache

כחלק מדרישות הבסיס של המטלה, נדרשנו לממש מנגנון זיכרון מטמון עבור ה-API החיצוני של מזג האוויר. כאשר משתמש נכנס למסך של פרטי אירוע, מוצג לו בין היתר מידע על צפי מזג האוויר באזור בו מתקיים האירוע.

המידע כולל :

תיאור מזג האוויר

טמפרטורה

תחושה בפועל

לחות

מימוש :

חלק זה ממומש בצד השרת באמצעות WeatherService. הקריאה הרלוונטית ב-API היא – GET /api/event/{id}/weather – כאשר מתקבלת בקשה למזג אוויר עבור אירוע מסוים, המערכת קודם שולפת את האירוע לפי ה-eventId, לאחר מכן היא לוקחת את שדה ה-Location של האירוע ואת תאריך האירוע – startDate ובונה ממנו בקשה ל-API חיצוני של מזג אוויר. יש שימוש ב-IHttpClientFactory לצורך שליחת בקשות HTTP ל-API החיצוני.

שימוש ב-API חיצוני :

במערכת נעשה שימוש ב-WeatherAPI.com. הבקשה נשלחת לפי מיקום האירוע ותאריך תחילת האירוע. ה-API מחזיר את המידע בפורמט של JSON, המערכת קוראת את ה-JSON, מוציאה ממנו את הנתונים הרלוונטיים ומחזירה ללקוח כ-WeatherDTO.

מנגנון Caching :

כדי למנוע קריאות מיותרות ל-API החיצוני, נוסף מנגנון Cache באמצעות IMemoryCache. המערכת שומרת את תוצאת מזג האוויר לפי מפתח ייחודי : weather_event_{eventId}_{eventDate} אם משתמש מבקש שוב את מזג האוויר עבור אותו אירוע – המערכת בודקת קודם האם המידע קיים כבר ב-Cache. אם כן, היא מחזירה את המידע מהזיכרון במקום לפנות שוב ל-API החיצוני. המידע נשמר ב-Cache למשך 10 דקות.

תהליך זרימת המידע במערכת :

המערכת בנויה לפי ארכיטקטורת שכבות, כך שכל שכבה אחראית על חלק מוגדר.

Client → Controller → Service → Repository → Database

המשתמש מבצע פעולה בצד הלקוח, כמו – צפייה באירועים או הרשמה ל-Sessions, הבקשה נשלחת אל ה-Controller המתאים בצד השרת, ה-Controller מעביר את הבקשה ל-Service בו נמצאת הלוגיקה העסקית, ה-Service משתמש ב-Repository כדי לגשת למסד הנתונים, וה-Repository עובד מול EventSystemContext שמייצג את החיבור לטבלאות במסד הנתונים.

בפרויקט קיימת הפרדה בין שכבות המערכת :

Client – כולל את קבצי צד הלקוח : JS, HTML, CSS
ManagementEventsAPI – כולל את ה-Controllers, Services, DTOs
Data – כולל את ה-Models, DbContext, Repositories

לוגיקה עסקית וולידציות :

בצד השרת מומשו הבדיקות הנדרשות בפרויקט שמטרתן לשמור על תקינות הנתונים במערכת.

- חסימת רישום כפול –
המערכת מונעת ממשתמש להירשם לאותו ה-Session יותר מפעם אחת.
לפני יצירת הרשמה חדשה, המערכת בודקת האם כבר קיימת רשומה עם אותו UserId ואותו SessionId. אם הרשמה כזו כבר קיימת – ההרשמה החדשה לא תתבצע.
- חסימת חפיפת זמנים -
המערכת מונעת ממשתמש להירשם ל-Session אם הוא כבר רשום ל-Session אחר שמתקיים באותו הזמן. המערכת שולפת את כל ה-Sessions שאליהם המשתמש כבר רשום ובודקת האם קיימת חפיפה בין זמני ה-Session החדש לבין אחד מהקיימים.
- תקינות תאריכי Session -
כאשר מוסיפים Session חדש לאירוע, המערכת בודקת שהוא אכן נמצא בתוך טווח הזמנים של האירוע. בודקת שה-Session : לא מתחיל לפני תחילת האירוע, לא מסתיים אחרי סיום האירוע, ושזמן הסיום לא לפני זמן ההתחלה.

מימוש קריאות API :

כל הקריאות ה-API הנמצאות בטבלת הדרישות מומשו במטלה.
הקריאות נבדקו דרך Swagger, וניתן גם לבדוק אותן באמצעות ה-Postman Collection.
צורף המסמך הרלוונטי.

צד הלקוח :

- מסך ראשי - מציג את רשימת האירועים הקיימים במערכת בכרטיסיות. לחיצה על כפתור ה-View Details תוביל למסך פרטי האירוע.
- מסך פרטי אירוע ולוח זמנים פנימי - מסך פרטי האירוע מציג את הפרטים של האירוע כולל שם, תיאור, תאריכי התחלה וסיום, מיקום וסוג האירוע. בנוסף, מוצג גם מזג האוויר הצפוי. מתחת לפרטים, מוצגת רשימת ה-Sessions השייכים לאותו האירוע מסודרת לפי שעות.
- מנגנון הרשמה ל-Sessions - במסך פרטי האירוע קיימת אפשרות להירשם ל-Session. המשתמש בוחר את עצמו לפי ה-UserId, והמערכת שולחת בקשת הרשמה לשרת. בצד השרת מתבצעות בדיקות ההרשמה הכפולה וחפיפת הזמנים לפני שמירת ההרשמה במסד הנתונים.

תוספות ופיצ'רים מתקדמים :

כפי שפורט בתחילת המסמך, הפיצ'רים שנבחרו הם :

1. ממשק מנהל ייעודי לניהול אירועים
2. מסך סטטיסטיקות
3. לוח זמנים אישי למשתמש

קישורים ופרטים חשובים :

קישור ל-GitHub : <https://github.com/oferavioz/ManagementEvents>

דף המנהל :

משתמש – *admin*

סיסמה – 1234

בדיקה נעימה!